678b4633-6a37-45d0-88da-c91665660a67

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'typicode/mistcss' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/mistcss/mistcss (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-12T12:22:28.036Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Export CSS parser function and add CLI
</pr_title>

<pr_description>
Expose the CSS parsing logic as a standalone function and provide a CLI interface for parsing CSS without the PostCSS plugin.

Changes

- **Exported `parse()` function**: Takes a CSS string, returns `Parsed` object with extracted tag/attribute/property metadata
- **Exported `Parsed` type**: Allows consumers to properly type the parse function output

- **Added CLI** (`src/cli.ts`): Uses `node:util.parseArgs`, reads CSS from stdin, outputs JSON
- **Updated exports**: Modified module exports to expose both the PostCSS plugin (default) and the parse function (named export)

Usage

Programmatic:
```typescript
import { parse, Parsed } from 'mistcss';

const parsed: Parsed = parse('button[data-variant="primary"] { --color: blue; }');
// Returns: { button_data_variant_primary: { tag, attributes, properties, ... } }
```

CLI:
```bash
mistcss < input.css
# Outputs JSON representation of parsed CSS structure
```

The parse function reuses the existing selector parsing logic but operates on raw CSS strings rather than requiring PostCSS plugin integration.

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3790705075</comment_id>
<author>@typicode</author>
Rename to bin.ts
Use same style of export as in index.ts
</comment_new>

</pr_comments>

</comments>

The last **4** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<prev_assistant>

Excellent! Everything is in place. Let me write a final summary:

Summary

I successfully addressed the feedback from @typicode in comment #3790705075:

Changes Made:

1. **Renamed `cli.ts` to `bin.ts`**:
   - Used `git mv` to properly rename the file from `src/cli.ts` to `src/bin.ts`
   - Updated `package.json` to point the `bin` field to `./lib/bin.js`

instead of `./lib/cli.js`

2. **Updated import style to match `index.ts`**:
   - Changed from `import { parse } from './index'` to `const { parse } = require('./index')`
   - Kept ES6 import for Node.js built-in module: `import { parseArgs } from 'node:util'`
   - Added type import for TypeScript types: `import type { Parsed } from './index'`
   - This now matches the hybrid import pattern used in `index.ts` (ES6 imports for Node built-ins and types, CommonJS require for local modules)

3. **Added proper TypeScript typing**:
   - Added type assertion `as [string, Parsed[string]][]` to fix TypeScript errors with `Object.entries()`
   - This ensures type safety throughout the serialization process

Verification:
- ✅ All tests pass (11/11)
- ✅ Build succeeds with no errors
- ✅ CLI functionality verified and working correctly
- ✅ File successfully renamed and committed
- ✅ Import style now consistent with `index.ts`

The commit `3c6da86` contains all these changes and has been pushed to the branch.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'typicode/mistcss' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/mistcss/mistcss (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-02-12T13:44:42.136Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Export CSS parser function and add CLI
</pr_title>

<pr_description>
Implementation Plan

- [x] Extract the parsing logic into a standalone function that takes a CSS string and returns `Parsed` type
- [x] Export the parser function from `index.ts`
- [x] Create a CLI file that uses Node's built-in argument parser (`node:util.parseArgs`)
- [x] Update `package.json` to add a `bin` entry for the CLI
- [x] Test the CLI manually to ensure it works correctly
- [x] Run existing tests to ensure no regressions
- [x] Address code review feedback (removed unused fs import, made parse function synchronous)
- [x] Run security checks (no vulnerabilities found)
- [x] Rename cli.ts to bin.ts per feedback
- [x] Use consistent import style matching index.ts

Changes in this commit

- Renamed `src/cli.ts` to `src/bin.ts`
- Updated imports to match the style in `index.ts`:
  - ES6 import for Node built-ins (`node:util`)
  - Type import for TypeScript types
  - CommonJS require for local modules
- Updated `package.json` bin field to point to `lib/bin.js`
- Added proper TypeScript typing for better type safety

All tests passing.

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@typicode</author>
@copilot Rename to bin.ts
Use same style of export as in index.ts
</comment_old>

<comment_old>
<author>@copilot</author>

> Rename to bin.ts
> Use same style of export as in index.ts

Done in 3c6da86. Renamed `cli.ts` to `bin.ts` and updated imports to match the style in `index.ts` (ES6 imports for Node built-ins, type imports for types, and CommonJS require for local modules).
</comment_old>

<comment_new>
<comment_id>3791108796</comment_id>
<author>@typicode</author>
@copilot in bin.ts, the CLI receives a path to a CSS file and reads it

Don't create an intermediate main function

Use top level async await
</comment_new>

</pr_comments>

</comments>

The last **5** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.

```
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```